PROJECT REPORT

On

“Simple ChatBot Using Python”

Submitted By:

Nayan Pannase (CM25035)

Guided By:

Mrs. Mayuri Getme

DEPARTMENT OF CSE (AI & ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY
MANAGEMENT AND RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2026-2027

© S.B.J.I.T.M.R Nagpur

INDEX

INDEX **i**

CHAPTER 1 INTRODUCTION ----------1

CHAPTER 2 METHODOLOGY 2 ----------4

CHAPTER 3 TOOLS/PLATFORMS 4 ----------10

CHAPTER 4 DESIGN & IMPLEMENTATION 5 ---------- 15

4.1 ALGORITHM

4.2 FLOWCHART

4.3 SOURCE CODE

CHAPTER 5 RESULT & DISCUSSION 9-11

5.1 OUTPUT

5.2 DISCUSSION

5.3 APPLICATION

CHAPTER 6 CONCLUSION 12

REFERENCES

1

3

CHAPTER 1
INTRODUCTION

1.1 Introduction

A chatbot is a program that communicates with a person through messages. This project develops a small desktop chatbot with an app-style graphical interface using Python and Tkinter. Instead of entering commands in a terminal, a user types into a message box, selects Send, and reads replies in a conversation panel. The application provides predefined responses in English, Hindi, and Marathi.

1.2 Background of the Project

Rule-based chatbots use written response rules rather than learning a language model. A message is normalized and checked against known phrases; a match selects a prepared answer. These systems are useful for fixed questions and introductory programming because their decisions can be understood and edited directly.

A graphical user interface (GUI) makes an application easier to approach for users who prefer buttons and windows. Tkinter is Python's standard GUI toolkit and includes controls for windows, text entry, buttons, labels, and scrollbars. It can be used without installing third-party packages in a standard Python desktop installation.

1.3 Problem Statement

A basic terminal exchange can be difficult to use for people unfamiliar with command-line programs. A fixed-language chatbot also limits access for users who prefer another language. The project addresses these needs by providing a simple designed desktop interface and predefined conversation rules in three languages, while preserving an ongoing interaction loop and a clear exit action.

1.4 Aim and Objectives

Aim: To develop a Python chatbot app that responds to predefined greetings and questions in English, Hindi, and Marathi, and continues to accept messages until the user chooses to exit.

1. Build a graphical chat window with a clear, friendly layout.
2. Provide a language selector for English, Hindi, and Marathi.
3. Respond to a set of predefined greetings and frequently asked questions in each selected language.
4. Normalize input to handle capitalization, extra spaces, and basic punctuation.

1.5 Scope and Limitations

The project supports one user at a time on a desktop computer. It matches known phrases and a limited number of greeting variations. It does not translate arbitrary text, understand every grammatical variation, learn from chats, store conversation history, or use an online AI service. The language selector determines the language of replies and interface labels; supported exit phrases are accepted across all three languages.

1.6 Proposed System

The app displays a welcome message and waits for a user message. The application checks whether the input is an exit phrase. Otherwise it normalizes the text and looks up a response using the selected language's phrase collection. A matching answer is added to the conversation panel; unmatched input receives a localized fallback. This cycle repeats until the user exits.

1.7 Importance and Intended Users

A compact multilingual interface demonstrates how software can make a familiar task easier to approach. A clear text box and Send button reduce the effort needed to begin a conversation, while the language selector gives users a direct choice of response language. The project is intended for students learning Python, instructors demonstrating GUI concepts, and users exploring a small set of common questions.

The project also illustrates the value of transparent behavior. The answers are prepared in advance and stored in the program, so users can see why a particular message receives a particular reply. This predictability is useful for fixed information, although it is less flexible than a chatbot that interprets open-ended requests. The simple input-processing-output structure also makes it possible to explain how a user action leads to a visible result.

Stage	What happens
Input	The user types a greeting or question and selects a response language.
Processing	The app normalizes the text, checks exit phrases, and searches that language's rules.
Output	The app adds a prepared answer or a helpful fallback to the conversation.

CHAPTER 2
METHODOLOGY

2.1 Development Approach

The work was organized into requirement definition, interface design, response design, implementation, and review. The interface was planned around the common chat-app pattern: a header, language control, scrollable conversation area, and message composer. Response rules were grouped by language so that the three phrase sets can be extended independently.

2.2 Requirement Analysis

Functional Requirements

- Open a desktop window with a welcome message.
- Accept typed messages and send them with a button or Enter key.
- Change the interface and response language between English, Hindi, and Marathi.
- Match predefined greetings and questions and display the corresponding response.
- Provide a localized fallback if a message is not recognized.
- Clear the conversation when requested.
- Recognize exit phrases and close further message entry.

Non-Functional Requirements

- Usability: readable text, clear controls, and visible feedback.
- Accessibility: keyboard Enter action and a resizable window.
- Portability: run on desktop systems that support Python 3 and Tkinter.
- Maintainability: separate response data from interface event handling.
- Privacy: process messages locally and do not transmit or save them.

2.3 Language and Conversation Design

Each language has a response dictionary mapping a supported phrase to a prepared response, plus a set of exit words. User text is trimmed and case-folded; common ending punctuation is ignored. The app first checks exit words, then exact phrase rules, then a small greeting variation rule. If no rule matches, it displays an appropriate fallback in the selected language.

The phrase collections are kept separate rather than merged into one list. This avoids ambiguity when the same spelling has different meanings and lets each response be reviewed in its own language. The user chooses a language explicitly; the program does not attempt to identify the language automatically or translate a message. A future change to the phrase collection can be made without redesigning the window.

2.4 Implementation Steps

1. Import Tkinter and define the three-language response and interface dictionaries.
2. Create the main window and apply its size, background, and theme.
3. Build the header, language selector, scrollable chat area, input box, Send button, and Clear chat control.
4. Connect button, Enter key, language selection, and clear actions to functions.
5. Normalize each message, select the matching reply, and add both messages to the chat panel.
6. Disable further entry after the user chooses an exit phrase.

2.5 Validation Scenarios

Representative interactions include selecting each language, sending a greeting, asking a supported question, entering text that is not in the phrase set, clearing the chat, and exiting with an English, Hindi, or Marathi exit phrase. The sample outputs in Chapter 5 illustrate these expected responses. The delivered Python file contains the complete runnable implementation.

2.6 Event-Driven Interaction

A desktop GUI responds to events rather than reading input from a sequential terminal prompt. Pressing Enter and clicking Send both call the same message handler. Selecting another language calls the label-update handler, while Clear chat resets the conversation panel and restores a welcome line. These actions keep the interface responsive while the main Tkinter event loop waits for the next user action.

After an exit phrase is recognized, the app displays a farewell and disables the message field and action controls. The window can then be closed using the normal operating-system window control. This gives the conversation a clear end state and prevents additional messages from being submitted accidentally.

CHAPTER 3
TOOLS / PLATFORMS

3.1 Software Requirements

Tool	Purpose
Python 3	Runs the chatbot and its interface logic.
Tkinter / ttk	Creates the desktop window, controls, and scrollbar.
Text editor or IDE	Used to view or modify chatbot_app.py.
Desktop operating system	Provides a windowing system for the graphical app.

3.2 Python and Tkinter

Python provides the application logic, phrase dictionaries, string normalization, and event-handling functions. Tkinter provides the graphical controls. The program uses a Text widget for conversation history, Entry for message input, Button controls for actions, a Combobox for language selection, and a Scrollbar for navigating the conversation.

3.3 Hardware Requirements

A standard laptop or desktop with keyboard, screen, and a supported Python installation is sufficient. No special hardware, network connection, database, or external package is required.

The app is designed for a single active conversation in one window. Its chat history remains visible for the current session only; closing the program discards it. This local, temporary behavior keeps the project simple and avoids collecting personal information. A production help tool would need a reviewed content policy and clear notice before storing any user messages.

3.4 Installation and Execution

1. Install Python 3 with Tkinter support. On Windows, the standard Python installer usually includes Tkinter.
2. Save chatbot_app.py in a convenient folder.
3. Open a terminal or command prompt in that folder.
4. Run python chatbot_app.py (or python3 chatbot_app.py).
5. Choose a language, type a message, and press Enter or click Send.

3.5 Using the Application

Select English, Hindi, or Marathi before starting a conversation. Type a supported greeting or question in that language and submit it with Enter or Send. The conversation panel shows the user message and the chatbot response in sequence. Use Clear chat to start over; use a supported exit phrase to end the session.

If a message receives the fallback, check that the selected language matches the wording and try one of the supported examples. The app accepts common punctuation at the end of a phrase, but it does not translate a message or infer a language automatically.

3.6 Common Setup Checks

Issue	Suggested check
Window does not open	Confirm Python 3 and its Tkinter support are installed.
Script cannot be found	Open the terminal in the folder that contains chatbot_app.py.
Message receives fallback	Use the selected language and a phrase from the supported examples.
Text display appears incorrect	Use a system font with Hindi and Marathi script support.

The chatbot runs locally and does not require an account or internet connection. The Hindi and Marathi script rendering depends on fonts available to the operating system; modern desktop systems typically provide script-capable fonts or fallback fonts.

CHAPTER 4
DESIGN & IMPLEMENTATION

4.1 System Architecture

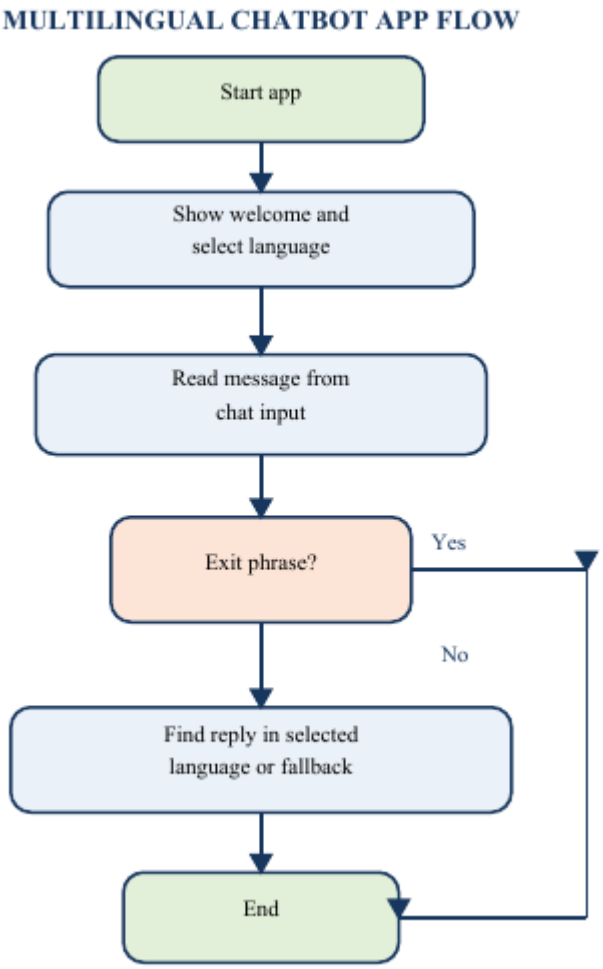
The application separates interface construction, language-specific content, and message processing. The window class creates the controls and handles events. Language dictionaries supply visible labels and responses. A reply function normalizes input, checks exits, finds a matching rule, and returns a fallback when necessary.

Module	Responsibility
Presentation	Displays the window, title, language control, conversation panel, and composer.
Language data	Stores predefined question-response pairs and localized interface labels.
Input handler	Reads typed text and responds to button or Enter events.
Reply selection	Normalizes text and finds an answer or fallback.
Exit control	Shows a farewell and disables further input when exit is requested.

4.2 Algorithm

1. Start the application and display the welcome message.
2. Wait for user input or a GUI event.
3. When a message is sent, trim spaces and normalize case and punctuation.
4. If the message is an exit phrase in any supported language, show a farewell and disable input.
5. Otherwise look up the message in the selected language's response dictionary.
6. If no exact match exists, check the limited greeting variation rules.
7. Display the selected answer or the localized fallback and wait for another message.
8. Repeat until the user exits or closes the application window.

4.3 Flowchart



4.4 Application Interface

The visual design uses a dark blue header, a soft gray-blue background, a white conversation surface, and a purple action button. Messages are labeled by speaker and displayed with spacing for readability. The language control sits above the conversation panel, while the input and Send button remain at the bottom of the window. The layout expands with the window and the conversation area includes a scrollbar.

The header presents the application identity and a short subtitle. Speaker labels use different accent colors to distinguish user messages from chatbot replies. The input control accepts text typed from the keyboard, and the scrollbar helps users revisit earlier messages in a longer exchange. Clear chat provides a simple way to begin a fresh interaction without restarting the program.

4.5 Code

```
def chatbot():
    print("Chatbot: Hello! Type 'exit' to end the chat.")

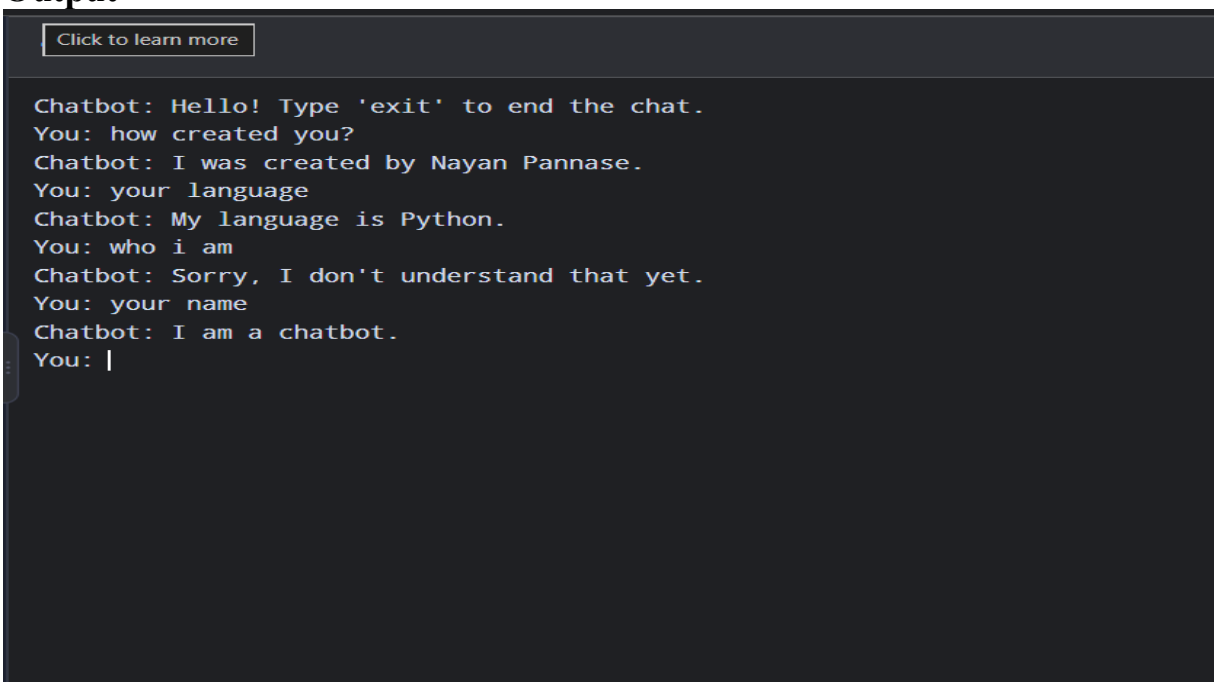
    while True:
        message = input("You: ").strip().lower()

        if message in ("exit", "quit", "bye"):
            print("Chatbot: Goodbye!")
            break
        elif message in ("hi", "hello", "hey"):
            print("Chatbot: Hello! How can I help you?")
        elif "how are you" in message:
            print("Chatbot: I'm fine, thank you! How are you?")
        elif "who are you" in message or "your name" in message:
            print("Chatbot: I am a chatbot.")
        elif "how created you" in message or "who created you" in message or "how were you
created" in message:
            print("Chatbot: I was created by Nayan Pannase.")
        elif "your language" in message or "which language" in message:
            print("Chatbot: My language is Python.")
        elif "help" in message:
            print("Chatbot: You can ask how I am, who I am, who created me, or what language I
use.")
        else:
            print("Chatbot: Sorry, I don't understand that yet.")


if __name__ == "__main__":
    chatbot()
```

CHAPTER 5
RESULT & DISCUSSION

5.1 Output



5.2 Discussion

The app combines a familiar graphical layout with a straightforward rule-based conversation loop. English, Hindi, and Marathi response collections are stored separately, allowing the content to be reviewed and expanded. The language selector also changes interface labels and the welcome and fallback messages. Exit words are recognized across the supported languages, so the user can finish the chat even after changing language.

The chatbot is not a translator and does not understand arbitrary phrasing. Users must choose the language and phrase the message close to one of the predefined examples. The program runs locally and does not save or transmit messages. Its limited scope makes it easy to explain while providing a starting point for a larger multilingual FAQ assistant.

Typical interactions include changing the response language, asking a known question, and trying an unsupported sentence. A known phrase produces its prepared answer; unknown text receives a localized fallback, and the chat remains active. Clear chat removes the visible exchange and returns a welcome message. Exit phrases in English, Hindi, or Marathi show a farewell and disable further entry.

Because the responses are rule-based, adding support for a new question requires adding a phrase and its answer to the correct language dictionary. Each answer should be checked for clarity and correctness in that language.

5.3 Applications

- Demonstrating GUI programming, event handling, dictionaries, and loops in Python.
- A small offline FAQ interface for a fixed set of common questions.
- A prototype for multilingual help desks or information kiosks after expanding and reviewing response content.
- A starting point for adding more languages, questions, accessibility controls, or persistent settings.

Potential use	Example role
Learning aid	Show how a Python program receives input and selects a reply.
College information prototype	Present reviewed answers to a small set of frequently asked questions.
Language practice	Let a learner compare short, prepared greetings across languages.
Kiosk concept	Demonstrate a local touch or keyboard interface for a fixed information topic.

These examples describe possible uses of the prototype. Before using it to provide official college information, the response text should be expanded, checked by the relevant department, and reviewed by fluent speakers. The current version is a classroom-scale demonstration rather than a maintained information service.

Classroom Demonstration

An instructor can demonstrate one conversation in each language, then ask learners to locate the response dictionary and add a new question-answer pair. Students can observe how the same interface continues to work when content is extended. This activity connects user interface design with dictionaries, string normalization, event handlers, and conditional decisions.

CHAPTER 6
CONCLUSION

The project develops a desktop chatbot app with an app-style Tkinter interface and predefined responses in English, Hindi, and Marathi. It presents a welcome message, accepts repeated user messages, responds to known greetings and questions, provides a localized fallback, and stops accepting input when the user enters an exit phrase.

The implementation demonstrates how user interface elements, language-specific data, text normalization, and event-driven control flow can work together. The chatbot meets its aim as a small multilingual application and can be extended with more carefully prepared response rules or additional interface features.

The project also provides practical experience in translating a simple algorithm into an interactive program. The rule set serves as the data layer, the reply method applies the decision logic, and the Tkinter widgets present the conversation. Keeping these roles understandable makes the code easier to explain, maintain, and adapt for another fixed-topic chatbot.

The completed prototype is appropriate for demonstrations and fixed-topic information. It gives the user control over the response language and session, while keeping the implementation small enough to inspect. Its main constraint is the finite response set: meaningful support for a broader audience would require more reviewed phrases, clear help text, and careful handling of variations in spelling and wording.

Learning Outcomes

- Apply Python dictionaries to organize multilingual response data.
- Connect GUI controls to event handlers and a persistent main loop.
- Use text normalization and conditional matching to select responses.
- Separate interface presentation from response content for maintainability.

Future Enhancements

- Add more questions and language variations after review by fluent speakers.
- Offer separate controls for conversation language and interface language.
- Add keyboard shortcuts, a theme selector, or adjustable text size.
- Provide optional conversation export with clear user control.
- Add categories such as college information, Python basics, and student services.

REFERENCES

1. Python Software Foundation, “tkinter - Python interface to Tcl/Tk,” Python 3 Documentation. https://docs.python.org/3/library/tkinter.html
2. Python Software Foundation, “The tkinter.ttk module,” Python 3 Documentation. https://docs.python.org/3/library/tkinter.ttk.html
3. Python Software Foundation, “Built-in Types - Text Sequence Type (str),” Python 3 Documentation. https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str
4. S. B. Jain Institute of Technology, Management and Research, Project Report Format, provided reference PDF. Used for chapter order and formatting requirements.

The report text and application behavior were prepared for this chatbot project. The supplied format document was used as a structural and formatting reference.